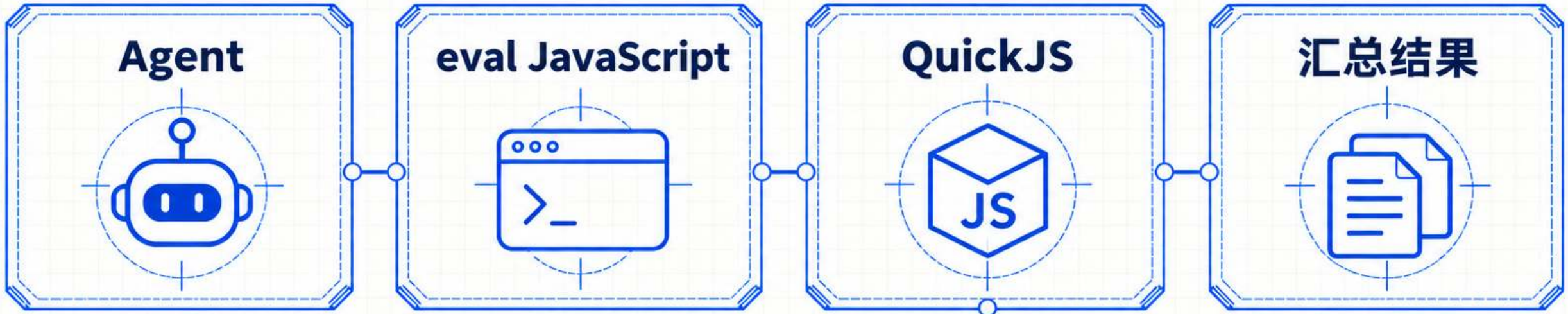
Deep Agents 实战

第15章：Interpreters — 让 Agent 用代码编排工具与数据



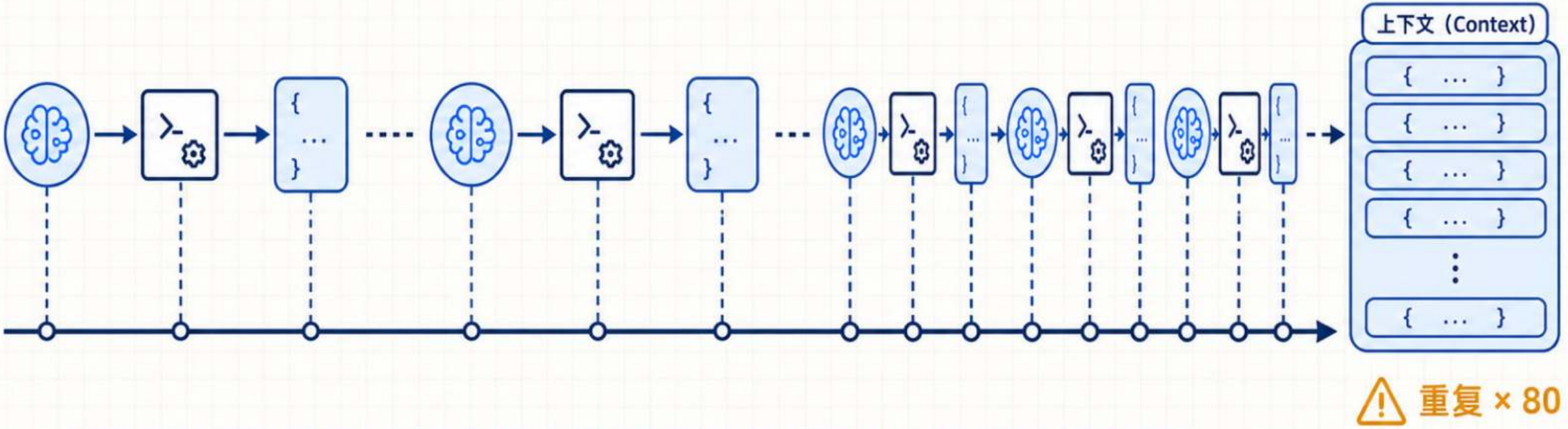
PTC tools | 连接到选定的外部能力

☆ 循环、筛选、聚合留在代码；只返回必要结果。

Beta API · 能力受限的进程内运行时，不是独立 Sandbox.

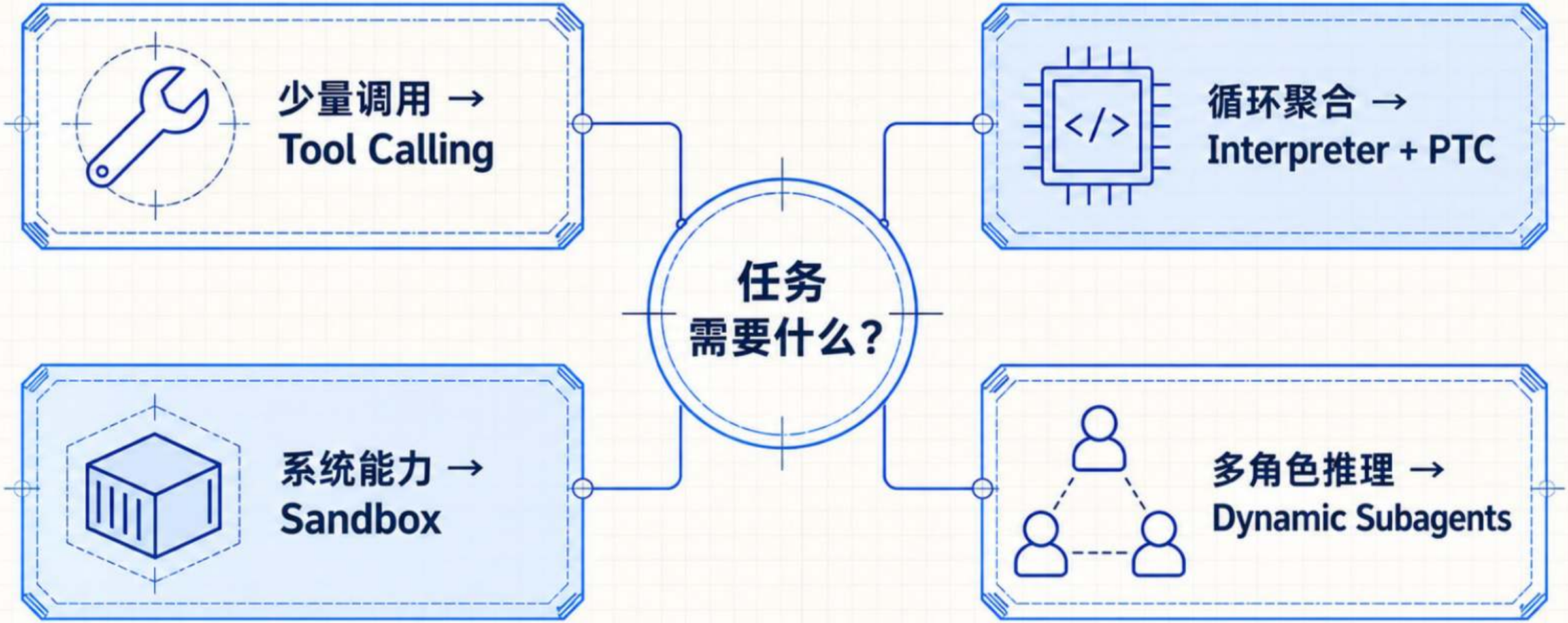
顺序工具调用的问题，不在工具本身

中间结果越多，模型上下文越拥挤



先看任务形状，再决定执行机制

调用规模、系统能力和推理角色是三个分岔点



QuickJS 是引擎，langchain-quickjs 是 Python 桥

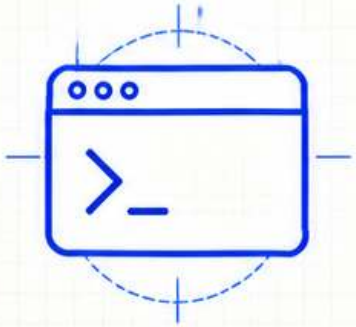
CodeInterpreterMiddleware 才是 Deep Agents 的接入点



deepagents[quickjs] → langchain-quickjs → CodeInterpreterMiddleware

接入 Interpreter：先锁版本，再选状态范围

1 环境：Python ≥3.11, Beta API



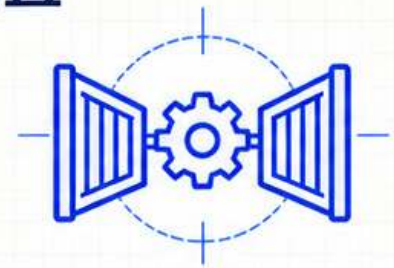
2 安装



```
uv add "deepagents[quickjs]" langchain-openai
```

章节参考版本 deepagents 0.7.8 / langchain-quickjs 0.3.5；复现实验需固定完整 lock。

3 配置



```
from langchain_quickjs import (
    CodeInterpreterMiddleware,
)
middleware = [
    CodeInterpreterMiddleware(mode="call")
]
```

模型 沿用已配置的工具调用模型；建议异步 ainvoke。

☆ **省略 mode 默认 thread；入门纯计算示例显式 call。**

此页为接入片段，未运行真实模型。

默认封闭，权限通过工具逐项打开

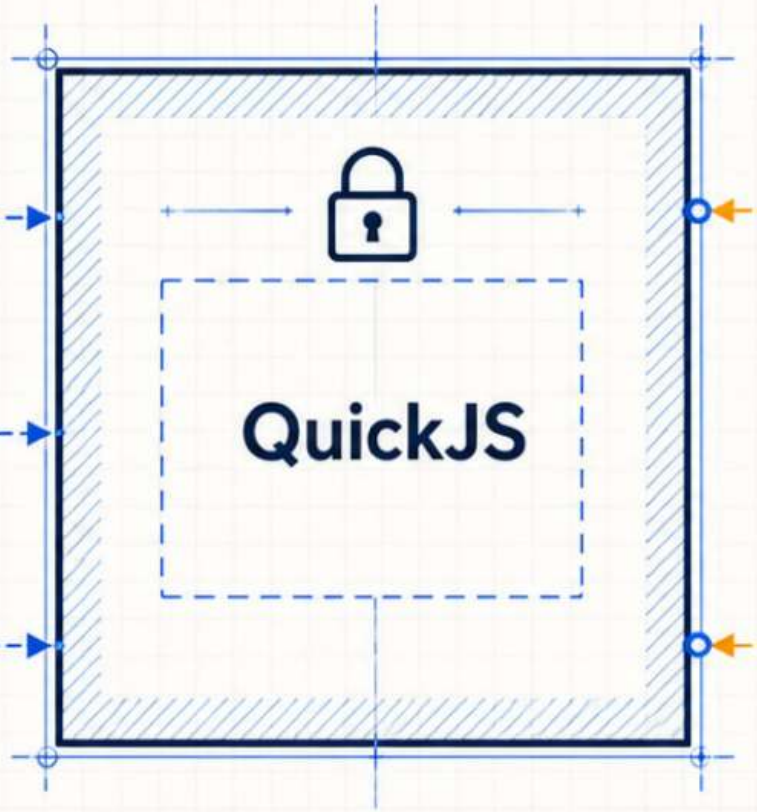
QuickJS 不是轻量版 Shell

默认可用：JavaScript、Promise、console

JS JavaScript

Promise

console



默认不可用：文件、网络、系统时间

不提供：Shell、包管理器、系统测试

PTC 两种白名单：工具对象与名称不要混淆

工具对象白名单

```
CodeInterpreterMiddleware(
    ptc=[lookup_order],
    mode="turn",
    max_ptc_calls=16
)
```

BaseTool 对象可只开放给解释器；
无需放入 agent tools。

名称白名单

```
tools=[lookup_order]
middleware=[CodeInterpreterMiddleware(
    ptc=["lookup_order"]
)]
```

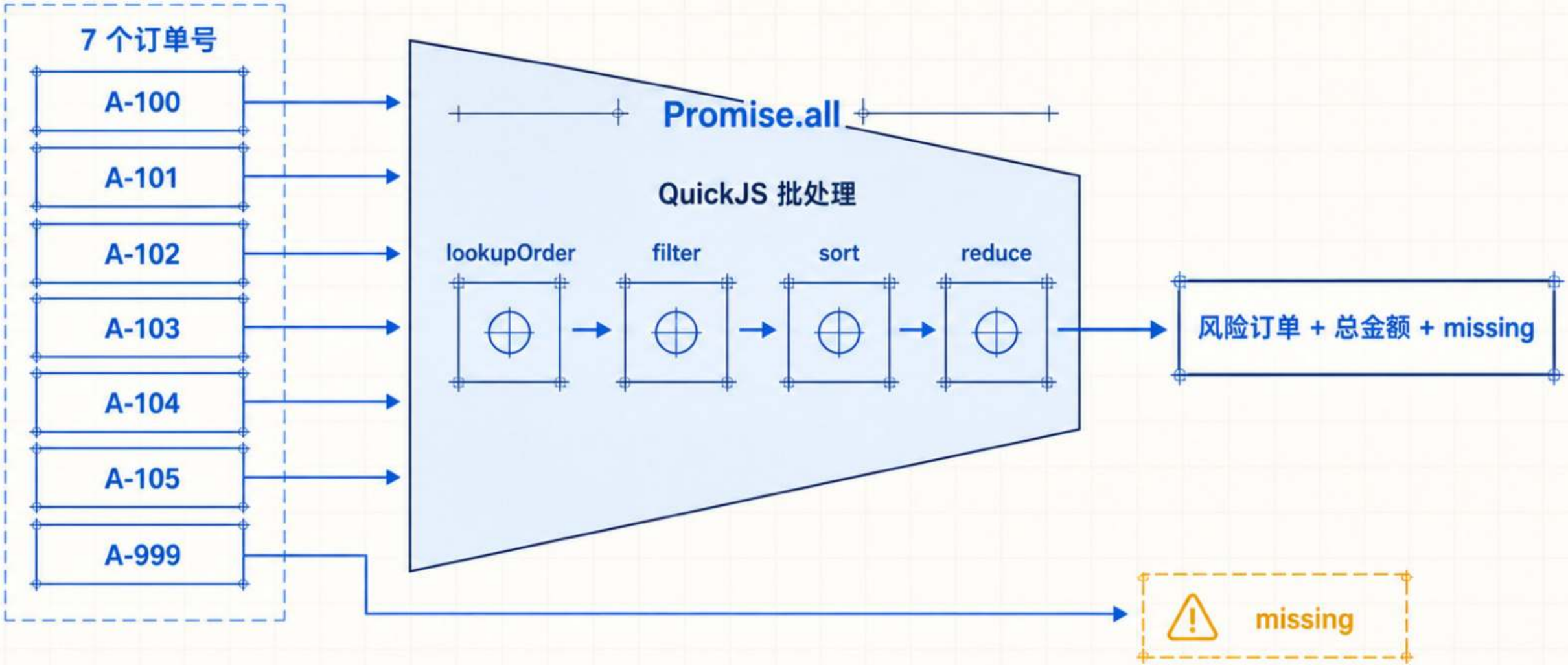
名称必须匹配 Agent tools；
此时普通工具路径仍可用。

Python `lookup_order` → JavaScript `tools.lookupOrder({order_id: "A-101"})`

函数名转 camelCase；参数保留 Tool Schema 的 order_id，不是 orderId。

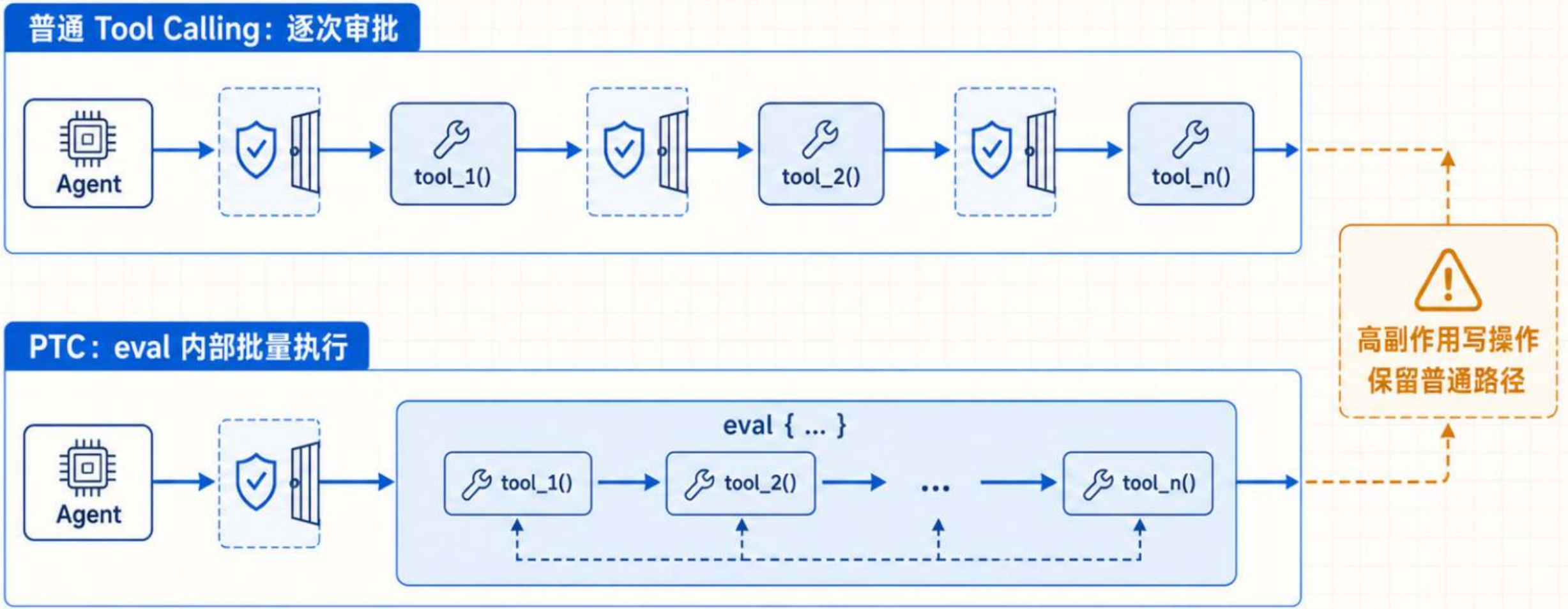
并行查询留在代码，模型只收汇总结果

Promise.all 覆盖全部输入，缺失项单独归类



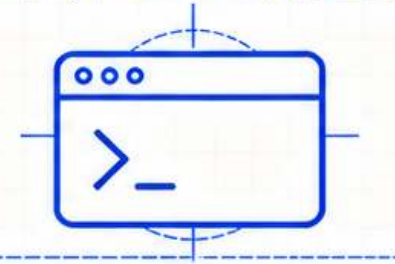
PTC 改变调用路径，也改变审批边界

interrupt_on 不会逐次覆盖 PTC 内部调用



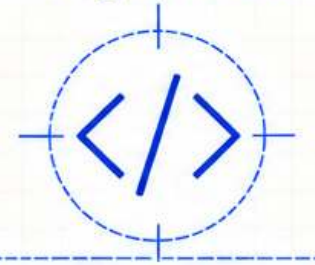
mode 控制状态范围，持久化需要额外配置

1 每次 eval 新上下文



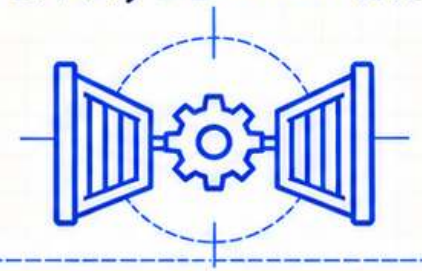
`mode="call"`

2 同一 Agent turn 内共享



`mode="turn"`

3 默认，跨 turn 恢复状态



`mode="thread"`

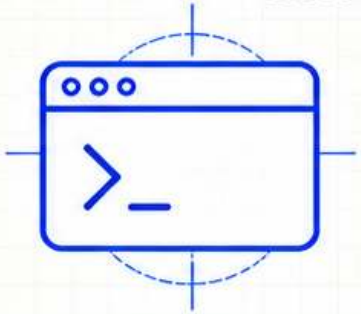
恢复 Snapshot → 当前 turn 多次 eval → 快照写回 Graph State → Checkpointer

- 同一 thread_id + 持久化 Checkpointer 才能跨重启恢复；MemorySaver 仅进程内实验。
- Snapshot 是版本相关的堆序列化机制；恢复能力与上限应实测，不要一概声称函数或类必定丢失。

⚠ 恢复 Snapshot 不会撤销外部工具副作用；超出 max_snapshot_bytes 可能丢弃快照。

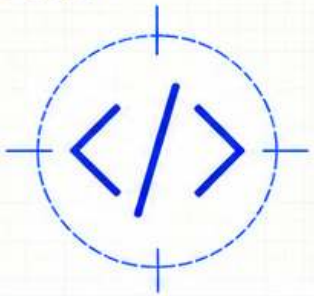
收紧能力、资源与状态：预算不是建议调用量

1 能力：只放必需的 BaseTool / 名称白名单



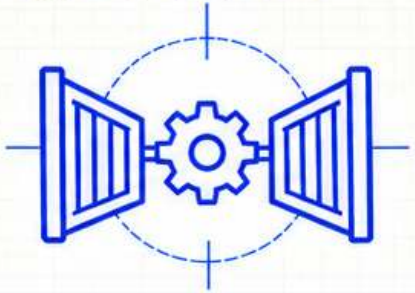
- 名称先注册在 agent tools
- subagents=False 可关闭 task

2 资源



- memory_limit=64*1024*1024
- timeout=5.0
- max_ptc_calls=16（本例主动收紧）

3 输出与状态



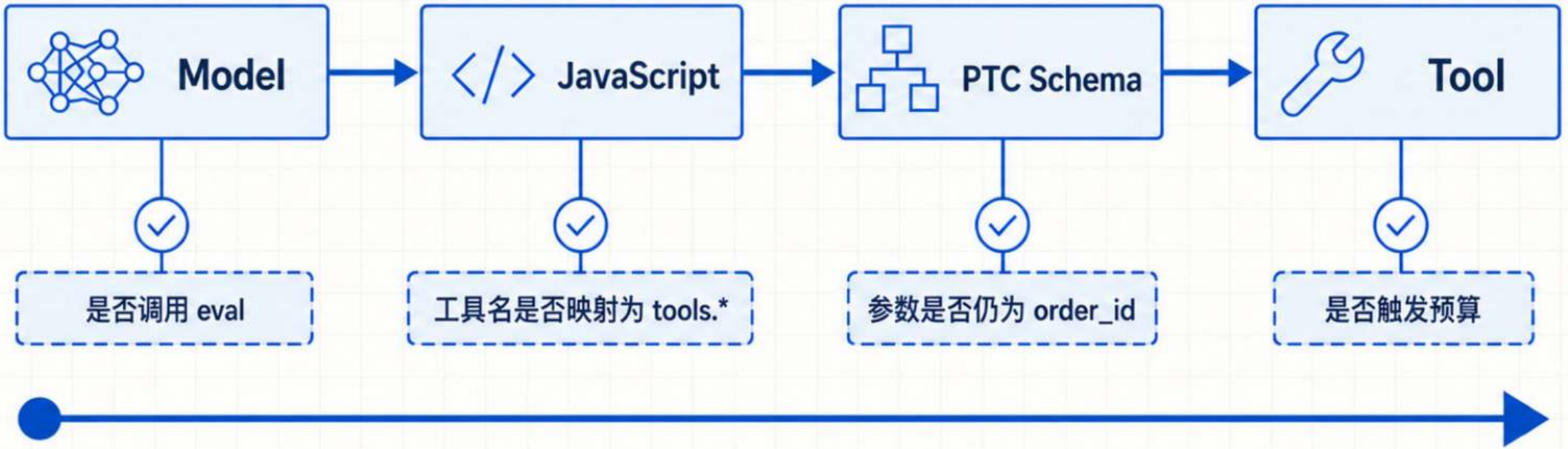
- max_result_chars=4000
- mode="call"
- capture_console=False（调试时再开启）

截断不是摘要：在 JavaScript 内先筛选与聚合。

☆ 工具自身继续鉴权、限额、校验。

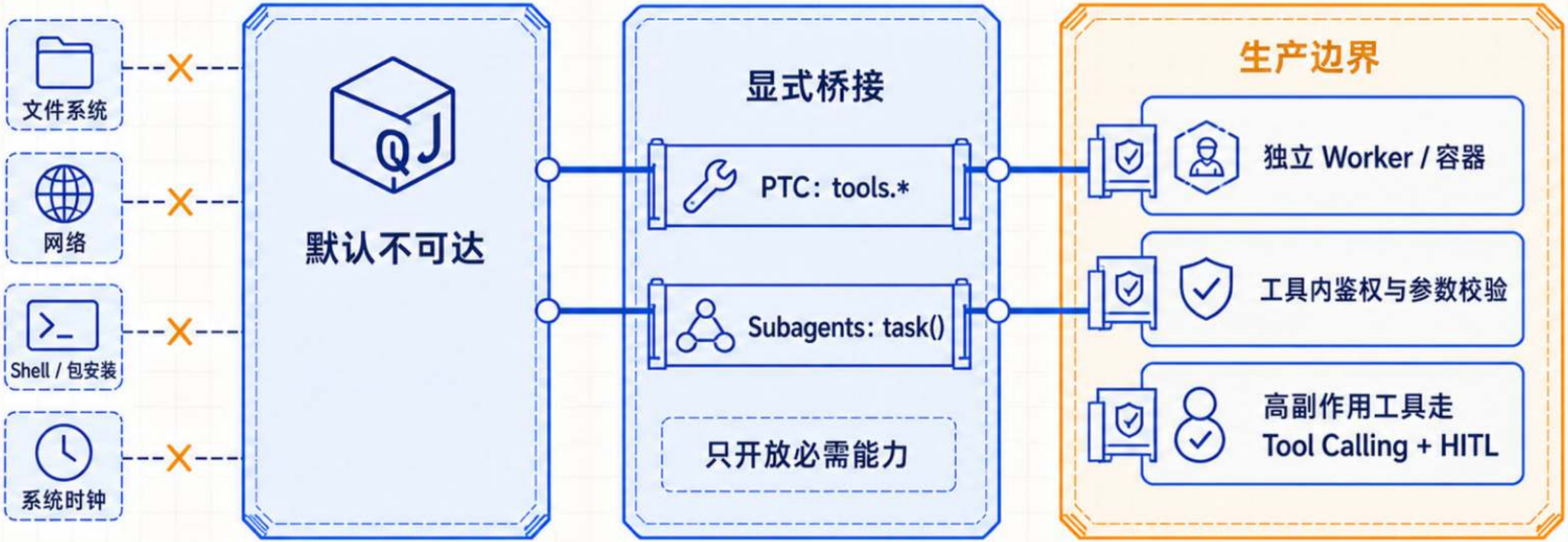
沿调用链排错，别只改 Prompt

Model → JavaScript → PTC Schema → Tool



QuickJS 隔离的是能力，不是宿主进程

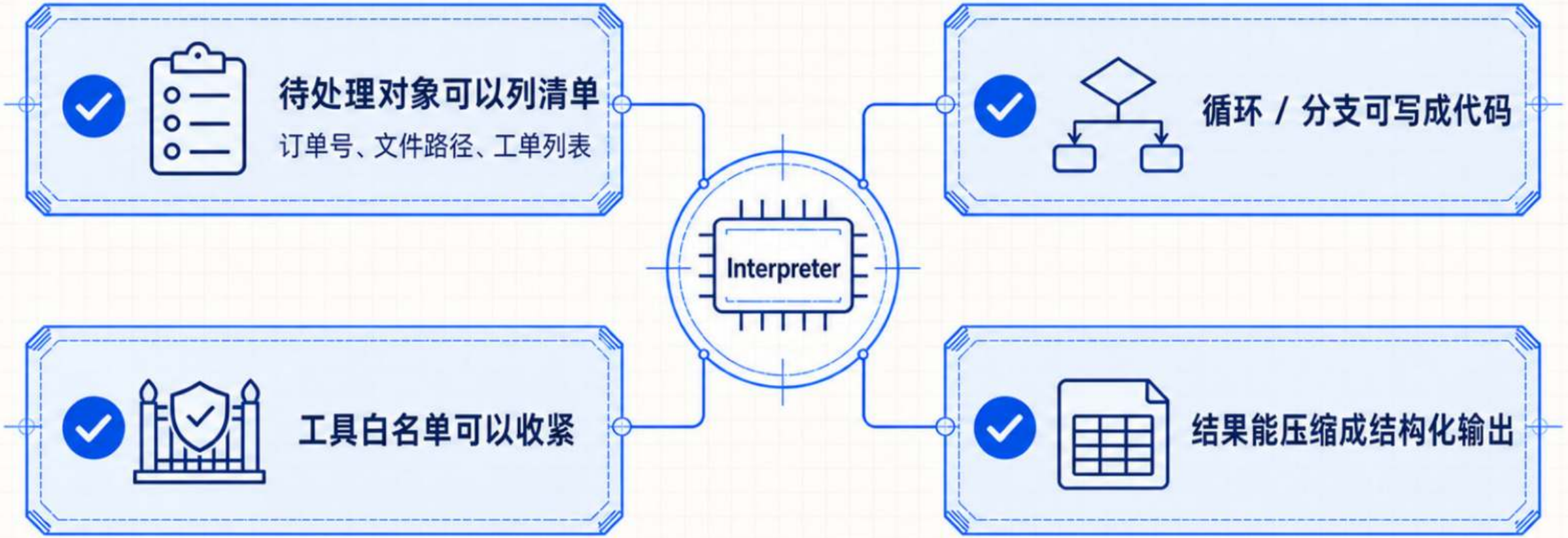
解释器与 Python 同进程运行；它不是 VM、容器或 Sandbox backend



PTC 工具能做什么，Interpreter 代码就能做什么

满足这四个条件，再用 Interpreter

任务能写成短程序，边界也能被明确限制



模型定目标，代码跑流程，工具守权限